

Class 6 Notes

Consistent Hashing — how distributed systems decide who owns a key, and how to survive servers joining and leaving

1. The Problem: Which Server Owns This Key?

We're building a distributed cache with three servers: A, B, C. Given a key like user:123, which server should store it? The obvious answer:

Naive placement

$\text{server} = \text{hash}(\text{key}) \% N$ — e.g. $\text{hash}(\text{"user:123"}) = 17$, and $17 \% 3 = 2 \rightarrow \text{Server C}$.

Simple, deterministic, uniform. Perfectly reasonable — until the cluster changes size. Ask yourself: what happens when we add Server D?

2. Why $\text{hash}(\text{key}) \% N$ Breaks

Every key's placement depends on N. Change N and almost every answer changes:

Key	hash	% 3 (before)	% 4 (after adding D)	Moved?
A	10	1 → Server B	2 → Server C	YES
B	15	0 → Server A	3 → Server D	YES
C	21	0 → Server A	1 → Server B	YES
D	37	1 → Server B	1 → Server B	no
E	44	2 → Server C	0 → Server A	YES
F	51	0 → Server A	3 → Server D	YES

Five out of six keys moved — and that ratio holds at scale: going from N to N+1 remaps roughly $N/(N+1)$ of all keys. Now picture a cache with 1 billion keys. Add ONE server, and hundreds of millions of keys suddenly point at machines that don't have them: a massive cache-miss storm, the database behind the cache takes the full read load, latency spikes, and a cascading failure is genuinely possible. All from adding capacity — the thing that was supposed to help.

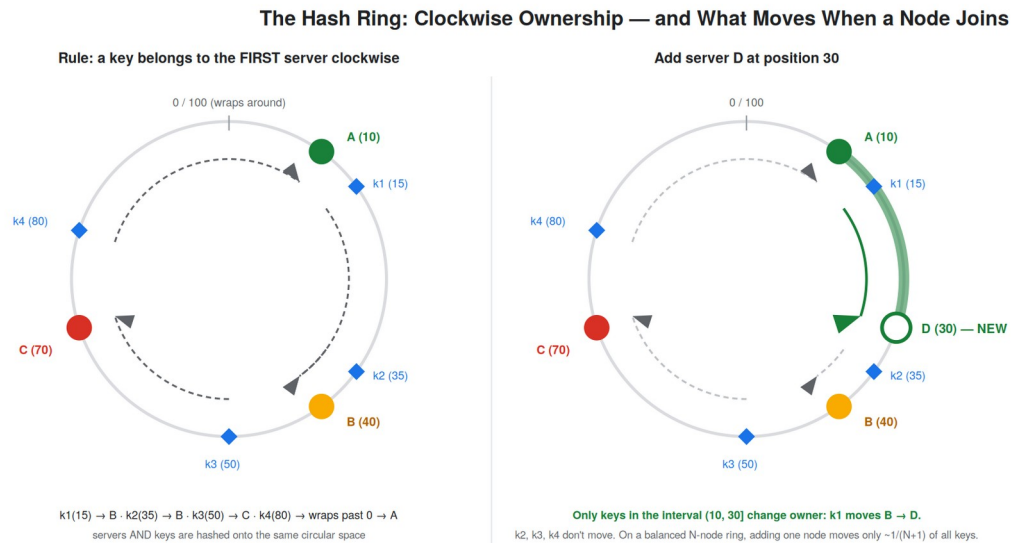
So we can state precisely what we want: a placement scheme where changing the number of servers moves only a small fraction of keys — ideally, only the keys that logically belong to the new (or dead) server. That property is what consistent hashing provides.

3. The Idea: Put Servers AND Keys on a Circular Hash Space

Two moves. First, treat the hash space as a circle — the largest value wraps around to 0. Second, hash the SERVERS onto that same circle, not just the keys. Then one rule decides everything:

The most important sentence of this class

A key belongs to the FIRST server encountered while moving CLOCKWISE around the ring from the key's position.



Walk the left ring: k1 at position 15 moves clockwise and first meets B(40) — so B owns it. k3(50) first meets C(70). k4(80) finds no server before the end of the space, wraps past 0, and lands on A(10) — the wraparound is why it's a ring, not a line.

4. The Payoff: Nodes Can Join and Leave Cheaply

- **Join (right ring above):** server D lands at position 30. The only keys whose clockwise-first-server changed are those in the interval (10, 30] — they used to reach B, now they hit D first. k1 moves; k2, k3, k4 are untouched. On a well-balanced ring of N nodes, the new node takes over $\sim 1/(N+1)$ of the keyspace: with 100 servers, adding one moves $\sim 1\%$ of keys — versus $\sim 99\%$ under modulo hashing.
- **Leave:** remove B and its keys (those in (10, 40]) simply continue clockwise to the next survivor, C. Nothing else moves. Only the departed server's range is affected.
- **Deterministic without a coordinator:** any client holding the ring metadata computes the owner of any key independently — no central lookup service on the request path.

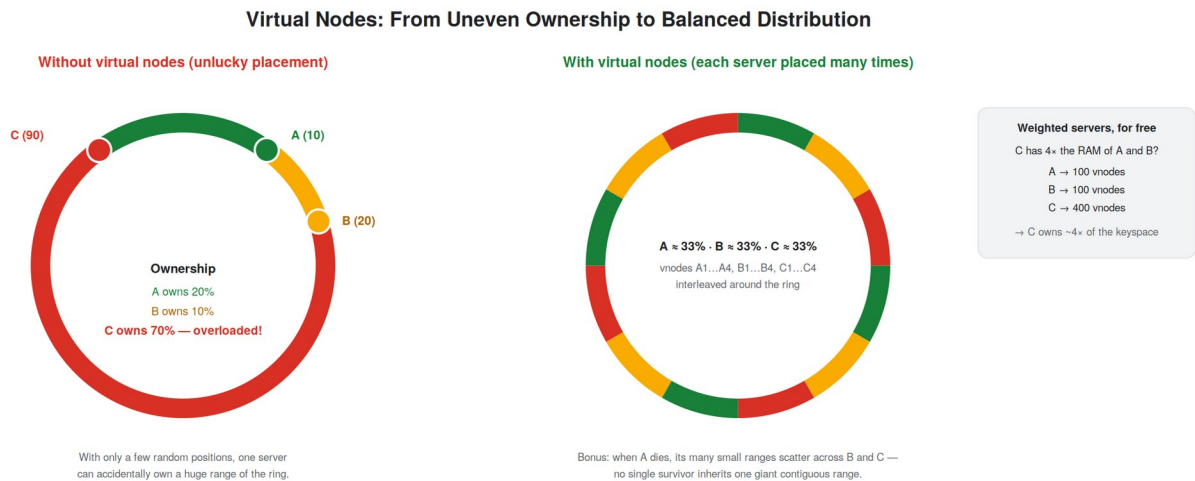
5. Under the Hood (Conceptually — No Code Needed)

The ring is just a sorted map from position → server. “First server clockwise” translates to: find the smallest server position $\geq$ the key's hash; if none exists, wrap to the smallest position overall. With a balanced-tree sorted map, lookup, node addition, and node removal are all $O(\log N)$ where N is the number of ring positions, with $O(N)$ space for the ring metadata. Two production details worth knowing:

use a stable, well-distributed hash function (MurmurHash-family, not a language's default hashCode — it must produce identical values on every machine, forever), and handle the rare position collision (two entries hashing to the same point) deliberately rather than silently overwriting.

6. The Catch — and the Fix: Virtual Nodes

With only a few servers, random positions can be brutally unlucky: place A at 10, B at 20, C at 90, and C owns 70% of the keyspace. Consistent hashing gave us minimal movement, but not balance.



The fix: put each physical server on the ring MANY times — as virtual nodes (vnodes), positioned by hashing “A-1”, “A-2”, “A-3”... With hundreds of positions per server, the law of large numbers evens out ownership. Two extra benefits come free:

- **Failure spreads instead of dumping:** when A dies, its many small ranges are inherited by many different survivors — no single neighbor absorbs one giant contiguous range and melts.
- **Weighted servers:** a machine with 4× the RAM simply gets 4× the vnodes and therefore ~4× the keyspace. Heterogeneous hardware handled with one number.

How many vnodes? There's no magic constant — anywhere from tens to 1000+ per server depending on cluster size and acceptable imbalance. More vnodes → smoother distribution, but more ring metadata and slightly costlier updates. It's a tuning knob, not a rule.

A distinction students blur — don't

Consistent hashing's promise is MINIMAL MOVEMENT when membership changes. BALANCED LOAD is a separate goal, delivered (statistically) by virtual nodes. Related, but different — and neither one guarantees protection from hot keys (Section 8).

7. Where You'll Meet It in Real Systems

System	How consistent hashing shows up
Distributed caches (Memcached-style clusters)	The classic use: clients hash keys onto a ring of cache servers; adding a cache node invalidates only $\sim 1/N$ of entries instead of everything.
Cassandra / Dynamo-style stores / Riak	Partition key $\rightarrow$ token on a ring $\rightarrow$ owning nodes. Cassandra's 'tokens' and 'vnodes' are exactly this class. (Recall Class 3: these are the PA/EL systems.)
Database sharding	user_id $\rightarrow$ shard via a ring, so resharding moves a slice of users, not all of them.
Load balancing (with care)	When the same client should keep hitting the same backend (session/cache affinity), hash-based routing works — but for plain stateless HTTP, round-robin or least-connections is often better. Consistent hashing is a tool, not a default.
Our course, next classes	Any time we say 'shard the trie' (Class 4) or 'partition the readings' (Class 2), this is the machinery underneath.

8. What Consistent Hashing Does NOT Solve

- **Hot keys.** If user:123 gets a million requests/sec, its owner melts no matter how perfect the ring is — one key always maps to one place. See the Q&A below for the standard fixes; this is the most common follow-up interviewers ask.
- **Replication.** The ring only names ONE owner. For fault tolerance, the standard extension: store each key on the next R distinct PHYSICAL nodes clockwise (skipping vnodes of the same machine) — first is primary, rest are replicas. Now a node failure is survivable: readers continue clockwise to a replica.
- **Failure detection and membership.** The ring answers 'who owns key K given the current member list' — it does NOT discover who is alive or make everyone agree on the member list. That's a coordination problem, solved by ZooKeeper/etcd/Consul or gossip protocols — and per Class 3, those are PC/EC systems, because a wrong membership map is worse than a briefly unavailable one.
- **What gets hashed.** Common misunderstanding: we hash the KEY (user:123) to place the data — we are not hashing the data/value itself.

The HLD framing that makes your answers stronger

Consistent hashing solves DATA PLACEMENT ("who owns this key?"). It does not solve CLUSTER COORDINATION ("who is alive, and does everyone agree?"). Real systems pair the two: ring for placement + ZooKeeper/etcd/gossip for membership + replication for failures. Saying this sentence in an interview instantly separates you from candidates who think the ring is the whole story.

9. Modulo vs Consistent Hashing — the Board Comparison

Property	hash(key) % N	Consistent hashing
Simplicity	Trivial	Slightly more machinery (ring + vnodes)
Deterministic	Yes	Yes
Add a server	$\sim N/(N+1)$ of keys move (almost all)	$\sim 1/(N+1)$ of keys move
Remove a server	Almost all keys move	Only the dead server's keys move
Load distribution	Uniform by construction	Good — with enough vnodes
Weighted / heterogeneous nodes	Not natural	Trivial via vnode counts
Lookup cost	$O(1)$	$O(\log N)$ on the sorted ring
Right tool when...	Cluster size is fixed and stable	Nodes join/leave/fail routinely

Honest takeaway: modulo hashing is excellent for a fixed cluster — it's simpler and $O(1)$. Consistent hashing earns its complexity exactly when membership is dynamic, which in large systems is always.

10. Advanced Aside: Rendezvous Hashing

Consistent hashing isn't the only answer. Rendezvous (Highest-Random-Weight) hashing computes $\text{score}(\text{key}, \text{node})$ for every node and picks the highest scorer — no ring at all. It gives excellent distribution, handles membership changes with the same minimal-movement property, and choosing replicas is natural (take the top R scorers). Its lookup is $O(N)$ in the number of nodes versus $O(\log N)$ for a ring, which matters for very large clusters. Learn the ring first — it's the mental model behind Cassandra and Dynamo — but knowing rendezvous exists is a strong interview signal.

11. Interview Q&A — Including the Follow-Ups That Actually Get Asked

Question	Strong answer
Why not just hash(key) % N?	Placement depends on N, so any membership change remaps ~everything — a cache-miss storm and database overload. Consistent hashing decouples placement from cluster size.
What happens when a node joins?	It takes ownership of only the interval between its predecessor and itself; only those keys move — $\sim 1/(N+1)$ of the keyspace on a balanced ring.
What happens when a node leaves/dies?	Its ranges flow clockwise to the next node(s); with vnodes, they scatter across many survivors; with replication, replicas serve immediately while data is rebuilt.
Why virtual nodes?	Three wins: statistical balance (no unlucky giant ranges), scattered failover load, and weighted capacity (bigger machine → more vnodes).

Question	Strong answer
Does it guarantee equal load?	No — twice no. Vnodes only balance KEYSACE statistically, and hot keys skew TRAFFIC regardless of keyspace balance.
How do you handle a hot key?	The ring can't help — attack the key itself: (1) replicate the hot key to R nodes and spread reads across them; (2) key salting — split it into user:123#1...#k landing on k different nodes, aggregate on read; (3) a small local/client-side cache for the top-K hottest keys (often the biggest win); (4) detect hot keys cheaply with approximate counters like count-min sketch (Class 4!) so the mitigation can be automatic; (5) for hot writes, shard the counter and merge asynchronously.
How do clients know who owns what?	The ring metadata (member list + positions) is small and distributed to clients or a routing layer; lookups are then local and deterministic. Keeping that metadata agreed-upon is the membership problem — ZooKeeper/etcd or gossip — not the ring's job.
How does replication work on a ring?	Walk clockwise from the key and take the next R DISTINCT physical nodes (skip vnodes of the same machine, and ideally spread across racks/zones): primary + R-1 replicas.
What data structure implements it, and complexity?	A sorted map of position → node: find the first entry $\geq \text{hash}(\text{key})$, wrapping to the first entry if none. $O(\log N)$ lookup/insert/delete, $O(N)$ space, N = ring positions (vnodes included).
Any subtle implementation gotchas?	Hash function must be stable and well-distributed across all machines and versions (Murmur-family, never a language's default hashCode); handle position collisions explicitly; changing the vnode count is itself a rebalancing event.
Is consistent hashing always the right sharding choice?	No. If you need RANGE queries (time-series scans, alphabetical ranges), hashing destroys locality — range-based partitioning fits better (recall Class 4's prefix shards, and their hot-shard trade-off). Hashing buys uniformity at the cost of order.

12. The One-Slide Summary

If you remember nothing else

1. $\text{hash}(\text{key}) \% N$ remaps ~everything when N changes — unacceptable for dynamic clusters.
2. Consistent hashing: hash servers AND keys onto a circular space; a key belongs to the first server clockwise.
3. Join/leave moves only the affected interval — $\sim 1/(N+1)$ of keys, not ~all of them.
4. Virtual nodes fix uneven ownership, scatter failover load, and enable weighted servers.
5. Consistent $\neq$ balanced $\neq$ hot-key-proof: three different problems (movement, distribution, skewed traffic) with three different tools (ring, vnodes, key replication/salting/caching).
6. The ring solves data placement only — membership and failure detection still need ZooKeeper/etcd/gossip, and durability still needs replication.

One sentence: consistent hashing maps keys and nodes onto a circular hash space so that when nodes join or leave, only a small portion of the keyspace is remapped — and virtual nodes make that ownership uniform.